

Universitatea din Bucuresti
Facultatea de Matematica si Informatica
Inteligența Artificială - Machine Learning

Project Report

Clasificarea imaginilor cu semnale radio

Chirițoiu Andra-Florentina

Grupa 234

Iunie 2026

Cuprins

1	Descrierea problemei si a datelor	2
2	Metodologie generala	2
3	Modelul 1: K-Nearest Neighbors baseline	4
3.1	Idee si preprocesare	4
3.2	Model si hiperparametri	4
4	Modelul 2: Linear SVM pe pixeli raw	5
4.1	Idee si preprocesare	5
4.2	Hiperparametri si rezultate	5
5	Modelul 3: Extra Trees Classifier	6
5.1	Idee	6
5.2	Rezultate	6
6	Modelul 4: HOG + Linear SVM	7
6.1	Idee	7
6.2	Rezultate	7
7	Modelul 5: HOG + Global Features + Hough + Linear SVM	8
7.1	Idee si caracteristici	8
7.2	Rezultate	8
8	Modelul 6: HOG + Global Features + Hough + Softmax	9
8.1	Idee	9
8.2	Rezultate	9
9	Modelul 7: XGBoost cu feature extraction extins	10
9.1	Idee	10
9.2	Rezultate	10
10	Modelul 8: XGBoost cu feature extraction + PCA	11
10.1	Motivatie	11
10.2	Rezultate	11
11	Modelul 9: CNN custom RGB + EDGE	12
11.1	Motivatie	12
11.2	Preprocesare	12
11.3	Arhitectura si antrenare	12
11.4	Rezultate	12
12	Modelul 10: DeepSignalCNN cu SE + ASPP - RGB + EDGE + FREQ	14
12.1	Motivatie	14
12.2	Preprocesare si antrenare	14
12.3	Rezultate si ensemble	14
13	Modele CNN avansate si ensemble final	15
13.1	GREEN + Sliding Window 1x7	15
13.2	2D-CNN cu Asymmetric Pooling si 1D-Profile-CNN	15
13.3	Ensemble final	15
14	Compararea tuturor metodelor	16
15	Concluzii	17

Rezumat

Acest raport documenteaza abordarea pentru competitia de clasificare a imaginilor cu semnale radio. Problema este una de clasificare multi-clasa, cu 5 clase. Au fost testate metode clasice de machine learning, metode cu extragere manuala de caracteristici si modele convolutionale antrenate de la zero. Sunt prezentate datele folosite, preprocesarea, augmentarile, reprezentarile de feature-uri, hiperparametrii testati, rezultatele pe setul de validare si matricile de confuzie. Cea mai buna solutie obtinuta a fost un ensemble intre doua modele 2D-CNN si un model 1D-Profile-CNN, cu acuratete pe validation de **0.7571** (toate valorile raportate în această documentație reprezintă acurateți calculate local; scorurile obținute pe Kaggle au fost, în general, mai mari, cu până la aproximativ **0.03**).

Raportul include toate abordările încercate, inclusiv variantele nereusite sau mai slabe.

1 Descrierea problemei si a datelor

Task-ul competitiei este clasificarea imaginilor de semnale radio in una dintre cele 5 clase. Fiecare exemplu este o imagine `.png`, iar fisierele `train.csv` si `test.csv` contin numele imaginilor si, pentru setul de antrenare, etichetele asociate.

Setul de date folosit are urmatoarea structura:

Tabela 1: Structura setului de date

Set	Numar imagini
Train	15500
Test	5500
Numar clase	5

Distributia claselor in setul de train este:

Tabela 2: Distributia claselor in train

Clasa originala	Numar imagini
1	3500
2	3000
3	3000
4	3000
5	3000

Pentru evaluarea locala, datele au fost impartite stratificat in 80% train si 20% validation:

Tabela 3: Split local train-validation

Subset	Numar imagini
Train local	12400
Validation	3100

In cod, etichetele au fost folosite intern ca valori 0, 1, 2, 3, 4, corespunzatoare claselor originale 1, 2, 3, 4, 5. Pentru modelele CNN, din cauza faptului ca prima clasa are mai multe exemple, au fost folosite ponderi de clasa in functia de pierdere:

Tabela 4: Ponderi de clasa folosite la modelele CNN

Clasa	Pondere
1	0.8857
2	1.0333
3	1.0333
4	1.0333
5	1.0333

2 Metodologie generala

Am pornit de la modele simple, pentru a verifica pipeline-ul de citire, preprocesare, antrenare, validare si generare a fisierului de submit. Ulterior, am crescut complexitatea prin feature extraction manual, modele ensemble clasice si retele

neuronale convolutive.

Fluxul general folosit a fost:

1. citirea imaginilor din fisiere `.png`;
2. impartirea datelor in train local si validation;
3. aplicarea preprocesarii specifice fiecarui model;
4. antrenarea modelelor;
5. alegerea hiperparametrilor pe validation;
6. analiza matricilor de confuzie;
7. generarea predictiilor pentru test.

Au fost testate urmatoarele familii de modele:

- modele pe pixeli raw: KNN, Linear SVM, Extra Trees;
- modele cu feature extraction manual: HOG, LBP, histograme, proiectii, FFT, connected components, Hough;
- modele liniare pe feature-uri manuale: Linear SVM si Softmax;
- XGBoost pe feature-uri extinse;
- XGBoost cu PCA;
- CNN-uri custom antrenate de la zero;
- ensemble-uri intre modele CNN.

Toate modelele finale respecta restrictia de a nu folosi date externe, modele preantrenate sau transfer learning.

3 Modelul 1: K-Nearest Neighbors baseline

3.1 Idee si preprocesare

Prima abordare a fost K-Nearest Neighbors, folosit ca baseline pentru verificarea intregului pipeline. Imaginile au fost:

- citite cu biblioteca PIL;
- convertite la grayscale;
- redimensionate la 64×64 ;
- normalizate prin impartirea valorilor pixelilor la 255.0;
- transformate in vectori 1D prin `flatten()`.

Pentru 64×64 , fiecare imagine devine un vector cu $64 \cdot 64 = 4096$ caracteristici.

3.2 Model si hiperparametri

KNN clasifica o imagine noua prin compararea cu imaginile din setul de train. Pentru fiecare imagine de validare, modelul calculeaza distanta fata de toate imaginile de antrenare, alege cei mai apropiati k vecini si returneaza clasa majoritara.

Au fost testate distantele L1 si L2:

$$d_{L1}(x, y) = \sum_i |x_i - y_i|$$

$$d_{L2}(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$$

Tabela 5: Rezultate KNN pe validation

Model	Image size	Metrica	k	Validation accuracy
KNN	64×64	L2	1	0.2100
KNN	64×64	L2	3	0.2155
KNN	64×64	L2	5	0.2200
KNN	64×64	L2	7	0.2158
KNN	64×64	L2	9	0.2216
KNN	64×64	L1	1	0.2010
KNN	64×64	L1	3	0.2148
KNN	64×64	L1	5	0.2223
KNN	64×64	L1	7	0.2132
KNN	64×64	L1	9	0.2158

Cea mai buna configuratie a fost $k = 5$, metrica L1, cu acuratete 0.2223. Rezultatul este slab deoarece modelul compara direct pixelii si nu invata structuri vizuale abstracte.

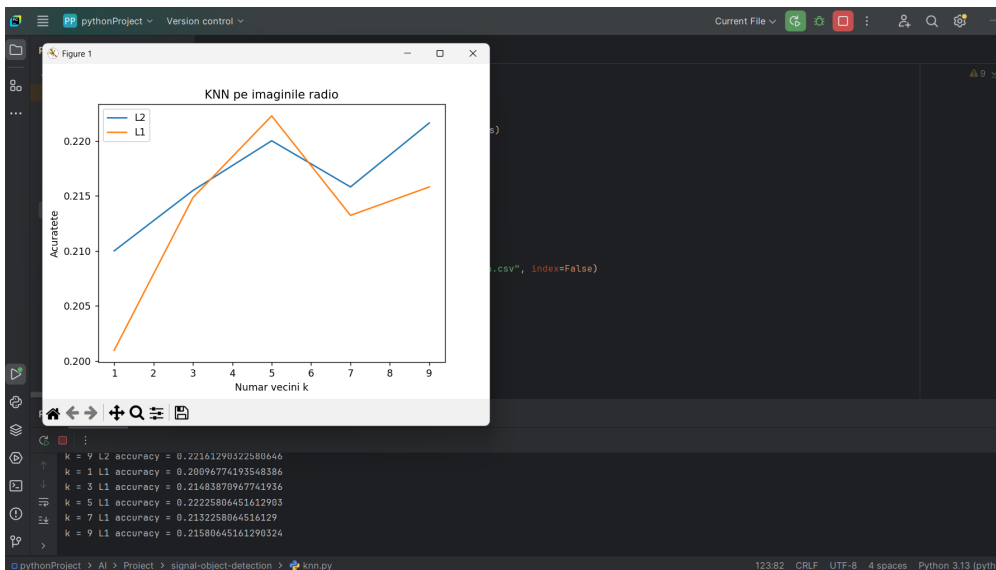


Figura 1: Rezultate de tuning pentru KNN: acuratetea pe validation pentru distantele L1 si L2 in functie de numarul de vecini.

4 Modelul 2: Linear SVM pe pixeli raw

4.1 Idee si preprocesare

Al doilea model a fost un Linear SVM aplicat pe vectorii de pixeli raw. Preprocesarea imaginilor a fost aceeași ca la KNN: grayscale, resize la 64×64 , normalizare in $[0, 1]$ si vectorizare. In plus, am folosit `StandardScaler`, antrenat doar pe train local si aplicat apoi pe validation.

4.2 Hiperparametri si rezultate

Parametrul principal testat a fost C , care controleaza compromisul dintre maximizarea marginii si penalizarea erorilor.

Tabela 6: Rezultate Linear SVM pe pixeli raw

Model	Image size	Features	C	Validation accuracy
Linear SVM	64×64	4096	0.001	0.2048
Linear SVM	64×64	4096	0.01	0.1984
Linear SVM	64×64	4096	0.1	0.1997
Linear SVM	64×64	4096	1	0.1984
Linear SVM	64×64	4096	10	0.1977

Cea mai buna configuratie a fost $C = 0.001$, cu acuratete 0.2048. Modelul nu a depasit KNN, ceea ce arata ca pixelii raw nu sunt o reprezentare suficienta pentru aceasta problema.

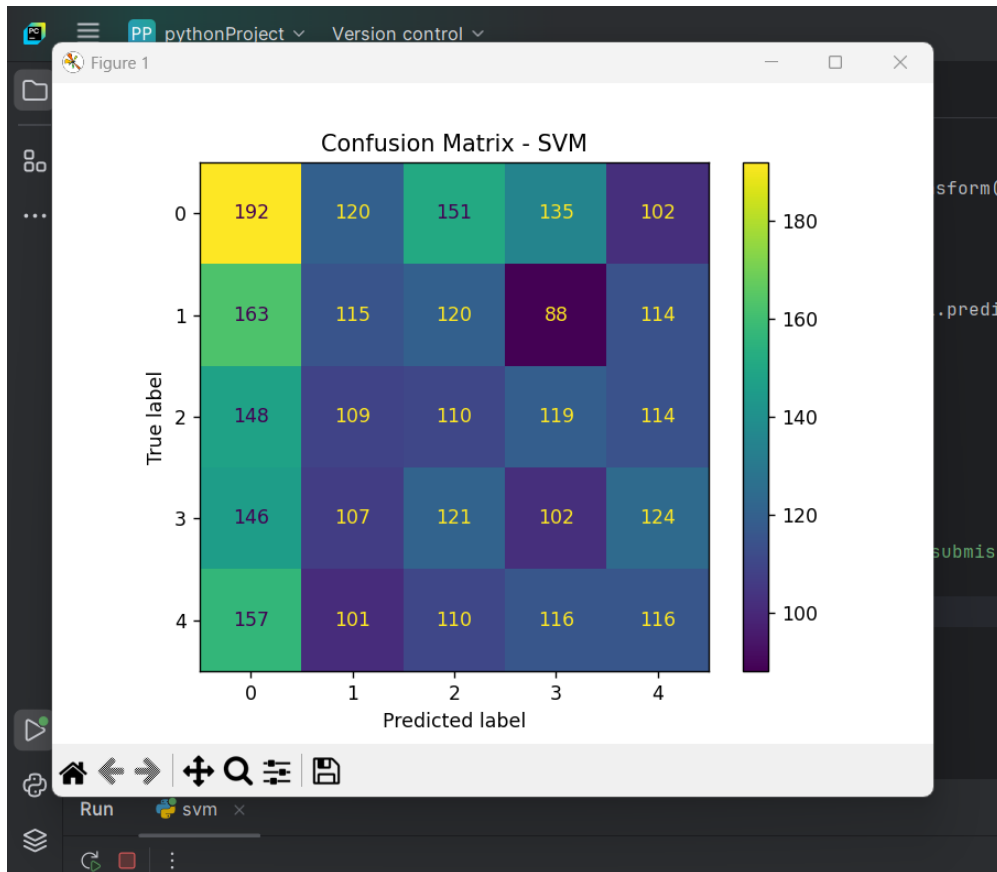


Figura 2: Matrice de confuzie pentru Linear SVM pe pixeli raw, calculata pe setul de validation.

5 Modelul 3: Extra Trees Classifier

5.1 Idee

Extra Trees este un model ensemble bazat pe mai multi arbori de decizie. Fiecare arbore invata reguli diferite, iar predictia finala se obtine prin vot majoritar. Modelul introduce mai multa randomizare decat un arbore clasic, alegand aleator anumite caracteristici si praguri de separare.

5.2 Rezultate

Tabela 7: Rezultate Extra Trees

Model	n_estimators	max_depth	Validation accuracy
Extra Trees	100	None	0.2600
Extra Trees	100	20	0.2535
Extra Trees	100	40	0.2684
Extra Trees	300	None	0.2665
Extra Trees	300	20	0.2587
Extra Trees	300	40	0.2558
Extra Trees	500	None	0.2616
Extra Trees	500	20	0.2600
Extra Trees	500	40	0.2645

Cea mai buna configuratie a fost `n_estimators=100`, `max_depth=40`, cu validation accuracy 0.2684. Extra Trees a fost mai bun decat KNN si Linear SVM pe pixeli raw, dar performanta ramane scazuta.

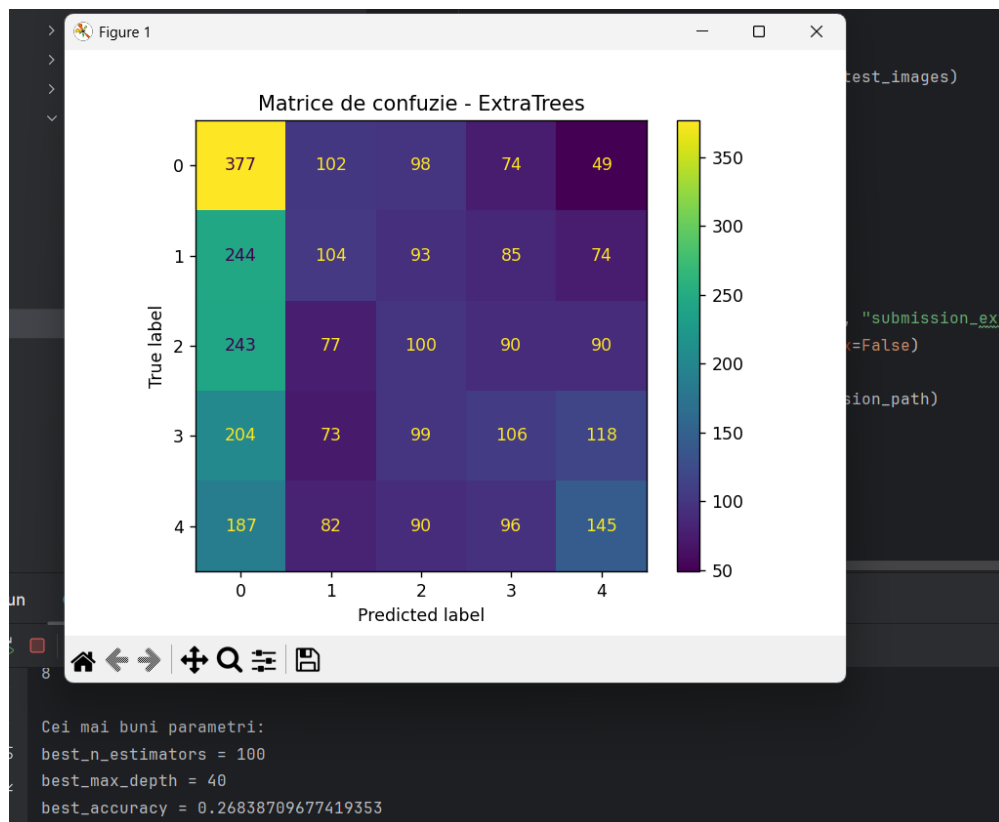


Figura 3: Matrice de confuzie pentru Extra Trees Classifier, calculata pe setul de validation.

6 Modelul 4: HOG + Linear SVM

6.1 Idee

Modelele anterioare au folosit direct pixelii imaginilor. Pentru a extrage caracteristici mai relevante, am folosit descriptorul HOG, adica Histogram of Oriented Gradients. HOG extrage informatii despre muchii, contururi si orientari locale.

Pipeline-ul a fost:

Image → grayscale → 64×64 → HOG → StandardScaler → LinearSVC.

Parametrii HOG folositi au fost:

Tabela 8: Parametri HOG

Parametru	Valoare
orientations	12
pixels_per_cell	(8, 8)
cells_per_block	(2, 2)
block_norm	L2-Hys
Image size	64×64

6.2 Rezultate

Tabela 9: Rezultate HOG + Linear SVM

Model	C	Validation accuracy
HOG + Linear SVM	0.00005	0.2981
HOG + Linear SVM	0.0001	0.2997
HOG + Linear SVM	0.0002	0.3026
HOG + Linear SVM	0.0003	0.3026
HOG + Linear SVM	0.0004	0.3003

Am ales $C = 0.0002$, deoarece obtine acelasi scor ca $C = 0.0003$, dar cu regularizare putin mai puternica.

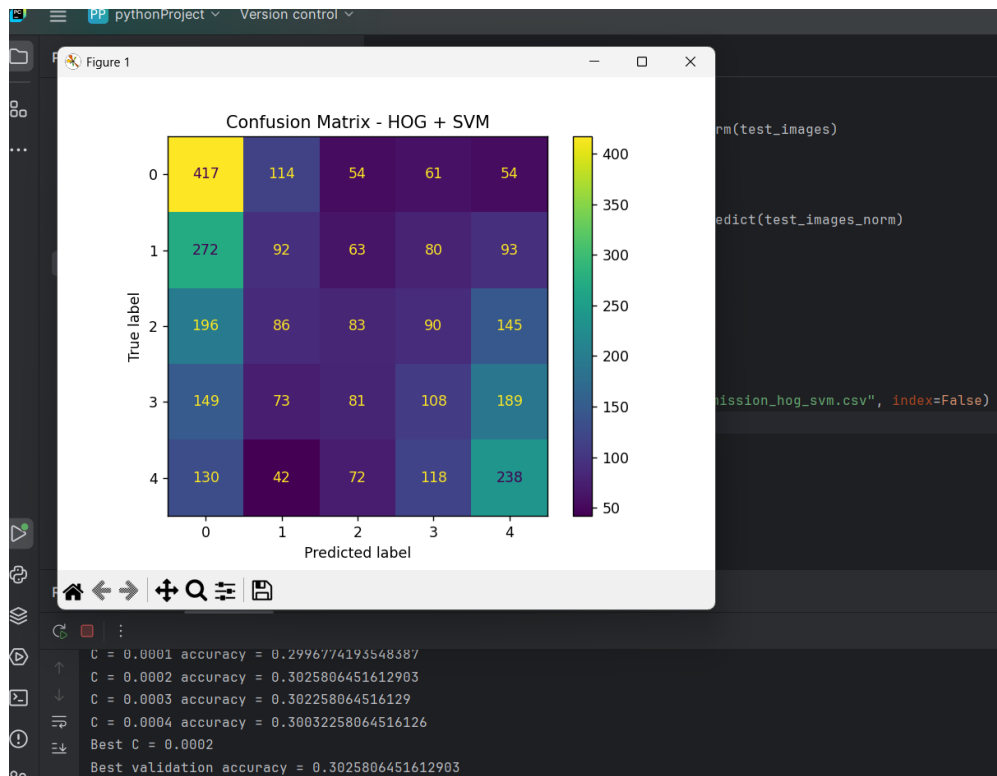


Figura 4: Matrice de confuzie pentru HOG + Linear SVM, calculata pe setul de validation.

7 Modelul 5: HOG + Global Features + Hough + Linear SVM

7.1 Idee si caracteristici

Dupa HOG simplu, am adaugat caracteristici suplimentare potrivite pentru semnale radio vizuale:

- statistici globale ale intensitatii pixelilor;
- distributia pixelilor luminosi pe randuri si coloane;
- numarul si aria componentelor conectate;
- caracteristici Hough pentru detectarea liniilor.

In aceasta configuratie, fiecare imagine a fost reprezentata prin 2449 caracteristici.

7.2 Rezultate

Tabela 10: Rezultate HOG + Global + Hough + Linear SVM

Model	class_weight	C	Validation accuracy
Linear SVM	None	$1 \cdot 10^{-6}$	0.2965
Linear SVM	None	$3 \cdot 10^{-6}$	0.3132
Linear SVM	None	$5 \cdot 10^{-6}$	0.3190
Linear SVM	None	$6 \cdot 10^{-6}$	0.3213
Linear SVM	None	$7 \cdot 10^{-6}$	0.3252
Linear SVM	None	$8 \cdot 10^{-6}$	0.3245
Linear SVM	None	$9 \cdot 10^{-6}$	0.3252
Linear SVM	None	$1.2 \cdot 10^{-5}$	0.3252
Linear SVM	balanced	$5 \cdot 10^{-6}$	0.3203
Linear SVM	balanced	$6 \cdot 10^{-6}$	0.3226
Linear SVM	balanced	$7 \cdot 10^{-6}$	0.3252
Linear SVM	balanced	$1 \cdot 10^{-5}$	0.3216

Cea mai buna configuratie a fost $C = 7 \cdot 10^{-6}$, fara `class_weight`, cu acuratete 0.3252.

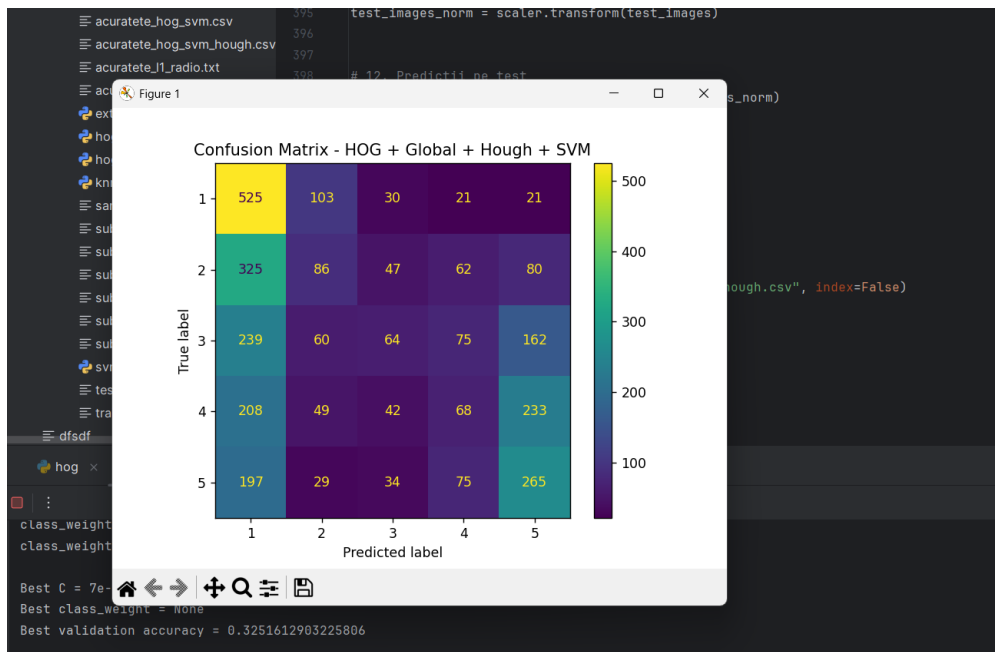


Figura 5: Matrice de confuzie pentru HOG + Global Features + Hough + Linear SVM, calculata pe setul de validation.

8 Modelul 6: HOG + Global Features + Hough + Softmax

8.1 Idee

Am testat si un clasificator Softmax, implementat prin `LogisticRegression`. Acesta este un model liniar multi-clasa, asemanator cu Linear SVM, dar optimizeaza o functie de pierdere diferita.

8.2 Rezultate

Tabela 11: Rezultate Softmax pe feature-uri manuale

Model	C	Validation accuracy
Softmax	0.0001	0.3342
Softmax	0.0003	0.3229
Softmax	0.0007	0.3197
Softmax	0.001	0.3206

Softmax a obtinut 0.3342 si a depasit varianta HOG + Global + Hough + Linear SVM.

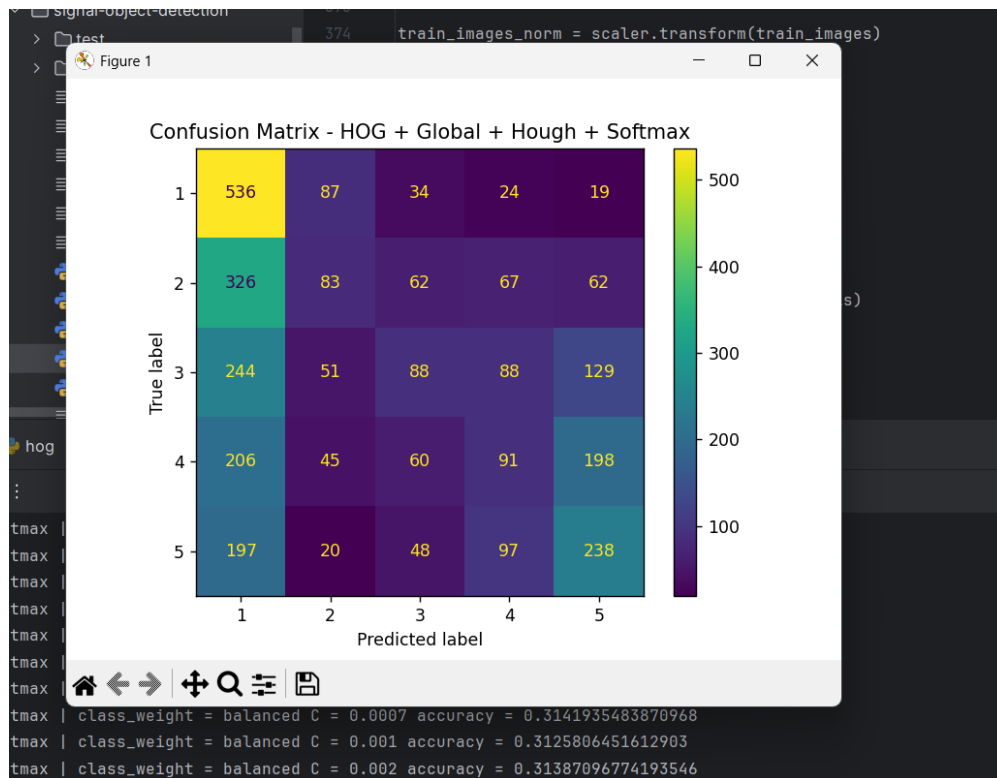


Figura 6: Matrice de confuzie pentru HOG + Global Features + Hough + Softmax, calculata pe setul de validation.

9 Modelul 7: XGBoost cu feature extraction extins

9.1 Idee

XGBoost a fost testat deoarece poate invata relatii neliniare intre caracteristici. Pentru acest model am folosit cea mai extinsa reprezentare manuala a imaginilor:

- HOG;
- LBP;
- histogram of intensity;
- proiectii pe randuri si coloane;
- FFT;
- connected components;
- Hough features.

Pentru XGBoost nu am aplicat **StandardScaler**, deoarece modelele bazate pe arbori nu sunt sensibile la scala caracteristicilor in acelasi mod ca modelele liniare.

9.2 Rezultate

Tabela 12: Rezultate XGBoost cu feature extraction extins

n_estimators	max_depth	learning_rate	subsample	colsample	reg_lambda	Accuracy
300	2	0.03	0.9	0.9	3	0.3758
300	3	0.03	0.9	0.9	3	0.3968
500	3	0.05	0.9	0.8	3	0.4094
500	4	0.03	0.8	0.8	5	0.4206

XGBoost fara PCA a fost cea mai buna metoda clasica, cu validation accuracy 0.4206.

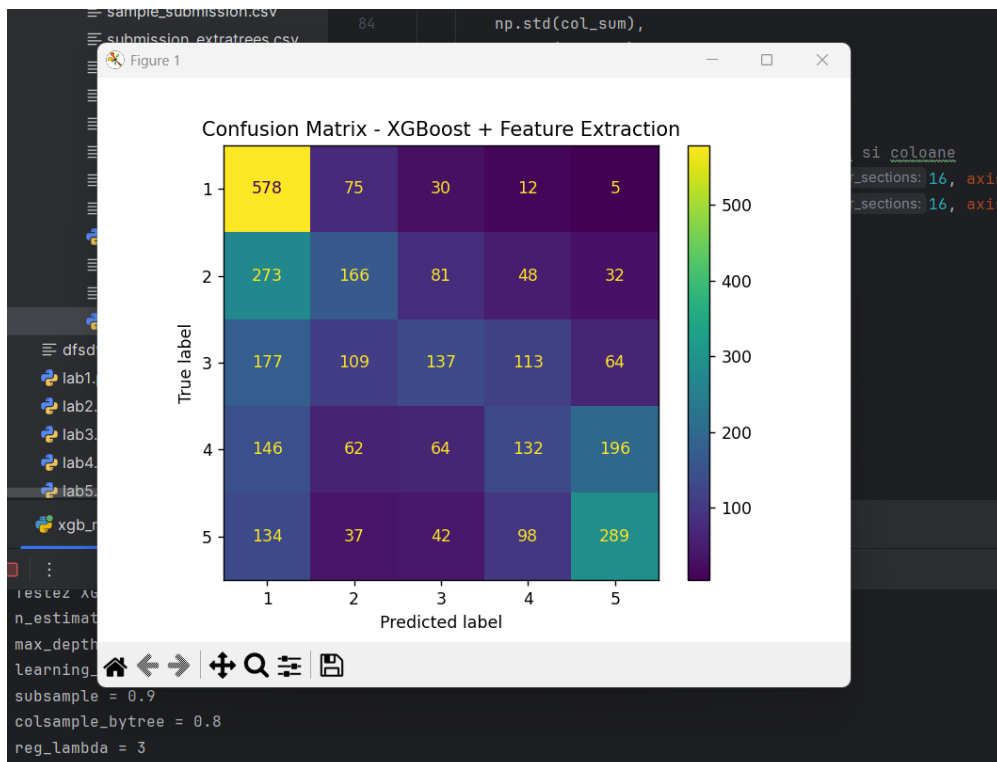


Figura 7: Matrice de confuzie pentru XGBoost cu feature extraction extins, calculata pe setul de validation.

10 Modelul 8: XGBoost cu feature extraction + PCA

10.1 Motivatie

Dupa XGBoost fara PCA, am testat reducerea dimensionalitatii cu PCA. Scopul a fost reducerea numarului de caracteristici, eliminarea zgomotului si obtinerea unei reprezentari mai compacte. Inainte de PCA am folosit **StandardScaler**, antrenat doar pe train local.

Pipeline-ul a fost:

Image → feature extraction extins → StandardScaler → PCA → XGBoost.

10.2 Rezultate

Tabela 13: Rezultate XGBoost + PCA

PCA	Numar componente	Explained variance	Best validation accuracy
50	50	0.5819	0.3513
100	100	0.6547	0.3458
200	200	0.7363	0.3577
0.95	798	0.9500	0.3355

Cea mai buna varianta a fost PCA cu 200 componente, cu acuratete 0.3577. Totusi, aceasta este mai slaba decat XGBoost fara PCA, ceea ce sugereaza ca PCA a eliminat si informatie utila pentru separarea claselor.

11 Modelul 9: CNN custom RGB + EDGE

11.1 Motivatie

Dupa modelele clasice, am implementat o arhitectura CNN proprie, antrenata complet de la zero. Scopul a fost sa folosesc un model care invata automat caracteristici vizuale relevante direct din imagini. Nu au fost folosite modele preantrenate sau date externe.

Au fost antrenate doua variante:

- model RGB, pe imaginile originale;
- model EDGE, pe reprezentarea grayscale + Sobel + Laplacian.

11.2 Preprocesare

Imaginile au fost redimensionate la 128×128 . Pentru modelul RGB, imaginea a fost pastrata in format RGB. Pentru modelul EDGE, cele trei canale au fost:

Tabela 14: Canale pentru reprezentarea EDGE

Canal	Descriere
1	image grayscale
2	magnitudinea Sobel
3	valoarea absoluta a filtrului Laplacian

Au fost folosite augmentari precum RandomHorizontalFlip, RandomAffine, ColorJitter, RandomErasing si Normalize.

11.3 Arhitectura si antrenare

Modelul `ScratchSignalCNN` contine blocuri Conv-BN-SiLU, blocuri reziduale, Dropout2d, Adaptive Average Pooling si un clasificator fully-connected. Setarile principale de antrenare au fost:

Tabela 15: Setari de antrenare pentru ScratchSignalCNN

Componenta	Valoare
Loss	CrossEntropyLoss cu label smoothing
Optimizer	AdamW
Learning rate	$3 \cdot 10^{-4}$
Weight decay	$1 \cdot 10^{-4}$
Scheduler	CosineAnnealingLR
Batch size	64
Max epochs	70
Patience	14
Image size	128×128

11.4 Rezultate

Tabela 16: Rezultate CNN custom RGB + EDGE

Model	Best epoch	Validation accuracy
ScratchSignalCNN RGB	60	0.6881
ScratchSignalCNN EDGE	52	0.6713
RGB + EDGE ensemble	-	0.6881

Ensemble-ul nu a imbunatatit modelul RGB, deoarece cea mai buna pondere gasita a fost 1.00 pentru RGB si 0.00 pentru EDGE.

Tabela 17: Matrice de confuzie - ScratchSignalCNN RGB

Clasa reala	Pred. 0	Pred. 1	Pred. 2	Pred. 3	Pred. 4
0	664	19	3	11	3
1	145	387	43	15	10
2	80	87	363	57	13
3	58	37	86	342	77
4	54	15	36	118	377

12 Modelul 10: DeepSignalCNN cu SE + ASPP - RGB + EDGE + FREQ

12.1 Motivatie

Am testat o arhitectura mai complexa, numita DeepSignalCNN, antrenata de la zero, care combina trei reprezentari:

- RGB: imaginea originala;
- EDGE: grayscale + Sobel + Laplacian;
- FREQ: reprezentare in domeniul frecventa.

Modelul foloseste blocuri convolutionale, Batch Normalization, activari SiLU, conexiuni reziduale, Squeeze-and-Excitation si ASPP. Fiecare model are 11.751.333 parametri antrenabili.

12.2 Preprocesare si antrenare

Pentru FREQ, imaginea a fost convertita la grayscale si a fost aplicata transformata Fourier 2D. Cele trei canale FREQ au fost magnitudinea logaritmica pentru frecvente joase, magnitudinea logaritmica pentru frecvente inalte si cosinusul fazei.

Setarile principale au fost:

Tabela 18: Setari DeepSignalCNN RGB + EDGE + FREQ

Componenta	Valoare
Image size	128×128
Batch size	48
Max epochs	80
Patience	18
Learning rate	$2 \cdot 10^{-4}$
Weight decay	$2 \cdot 10^{-4}$
Label smoothing	0.08
GPU	NVIDIA A100-SXM4-80GB

12.3 Rezultate si ensemble

Tabela 19: Rezultate DeepSignalCNN pe reprezentari multiple

Model	Best epoch	Validation accuracy
RGB	77	0.6987
EDGE	57	0.6800
FREQ	26	0.2829
RGB + EDGE + FREQ ensemble	-	0.6997

Temperaturile folosite pentru temperature scaling au fost 1.0504 pentru RGB, 0.8763 pentru EDGE si 1.4859 pentru FREQ. Cea mai buna combinatie de ponderi a fost:

$$0.85 \cdot p_{RGB} + 0.05 \cdot p_{EDGE} + 0.10 \cdot p_{FREQ}.$$

Tabela 20: Matrice de confuzie - RGB + EDGE + FREQ ensemble

Clasa reala	Pred. 0	Pred. 1	Pred. 2	Pred. 3	Pred. 4
0	685	13	1	1	0
1	160	400	20	13	7
2	98	73	359	47	23
3	80	27	76	349	68
4	60	18	26	120	376

13 Modele CNN avansate si ensemble final

Aceasta parte grupeaza ultimele variante testate, pentru a evita repetitia si pentru a pastra raportul compact. Toate modelele din aceasta sectiune au fost antrenate de la zero, fara date externe, fara modele preantrenate si fara transfer learning. Scopul a fost adaptarea arhitecturii la structura imaginilor de semnal radio, unde apar linii, benzi luminoase si distributii diferite pe axa verticala si orizontala.

13.1 GREEN + Sliding Window 1x7

Prima imbunatatire peste ensemble-ul RGB + EDGE + FREQ a fost folosirea canalului verde si a unei ferestre locale 1×7 . Cele trei canale de intrare au fost: canalul verde original, media locala pe fereastra 1×7 si diferenta dintre canalul verde si media locala. Modelul a fost antrenat 90 de epoci, cu learning rate initial $2 \cdot 10^{-4}$, batch size 48 si patience 20. Cel mai bun rezultat a fost **0.7139** validation accuracy.

Tabela 21: Rezultate GREEN + Sliding Window 1x7

Epoca	Validation accuracy
10	0.4687
20	0.6506
30	0.6816
49	0.7052
75	0.7106
90	0.7139

Tabela 22: Matrice de confuzie - GREEN + Sliding Window 1x7

Clasa reala	Pred. 0	Pred. 1	Pred. 2	Pred. 3	Pred. 4
0	670	25	3	2	0
1	143	409	33	11	4
2	88	74	389	38	11
3	66	28	89	359	58
4	54	18	30	112	386

13.2 2D-CNN cu Asymmetric Pooling si 1D-Profile-CNN

A doua imbunatatire majora a fost un 2D-CNN cu pooling asimetric. Acesta pastreaza mai bine structurile directionale ale semnalului decat pooling-ul simetric. In paralel, am testat si un 1D-Profile-CNN, care foloseste profiluri extrase din imagine, precum suma sau media pixelilor pe randuri si coloane. Modelul 1D a fost slab individual, dar a adus o contributie mica in ensemble.

Tabela 23: Rezultate 2D-CNN si 1D-Profile-CNN

Model	Parametri	Validation accuracy
2D-CNN asymmetric pool	12.044.069	0.7558
1D-Profile-CNN	5.707.077	0.3255
2D-CNN + 1D-Profile-CNN ensemble	-	0.7561

Cea mai buna combinatie intermediara a fost 90% 2D-CNN si 10% 1D-Profile-CNN. Matricea de confuzie a aratat ca modelul recunoaste foarte bine clasa 0, dar confunda in continuare clasele apropiate 3 si 4.

13.3 Ensemble final

Pentru submitul final am pastrat modelul 2D-CNN principal si am adaugat un al doilea 2D-CNN antrenat cu seed diferit, SEED_EXTRA=2026. Al treilea model din ensemble a fost 1D-Profile-CNN, pastrat pentru informatie complementara. Inainte de combinare am aplicat temperature scaling pentru calibrarea probabilitatilor.

Tabela 24: Modele folosite in ensemble-ul final

Model	Val. acc. individuala	Temperature	Pondere finala
2D-CNN old	0.7542	1.1755	0.65
2D-CNN extra seed2026	0.7435	1.2004	0.10
1D-Profile-CNN	0.2323	1.8802	0.25

Predictia finala a fost calculata ca medie ponderata:

$$p_{final} = 0.65 \cdot p_{2D-old} + 0.10 \cdot p_{2D-extra} + 0.25 \cdot p_{1D-profile}.$$

Rezultatul final pe validation a fost **0.7571**, cel mai bun scor local obtinut in proiect.

Tabela 25: Matrice de confuzie - ensemble final

Clasa reala	Pred. 0	Pred. 1	Pred. 2	Pred. 3	Pred. 4
0	661	23	10	5	1
1	118	455	15	8	4
2	54	76	446	16	8
3	35	28	107	393	37
4	21	24	50	113	392

Tabela 26: Classification report - ensemble final

Clasa	Precision	Recall	F1-score	Support
0	0.7435	0.9443	0.8320	700
1	0.7508	0.7583	0.7546	600
2	0.7102	0.7433	0.7264	600
3	0.7346	0.6550	0.6925	600
4	0.8869	0.6533	0.7524	600
Accuracy	0.7571			
Macro avg	0.7652	0.7509	0.7516	3100
Weighted avg	0.7645	0.7571	0.7542	3100

Cele mai importante confuzii au fost: clasa 3 prezisa ca 2 in 107 cazuri, clasa 4 prezisa ca 3 in 113 cazuri si clasa 2 prezisa ca 1 in 76 cazuri. Aceste erori sunt explicabile prin asemanarea vizuala dintre clase consecutive. Pentru test au fost folosite 8 predictii TTA pentru fiecare componenta, iar fisierul final generat a fost **submission_LAST_BEST.csv**.

14 Compararea tuturor metodelor

Tabela 27: Rezumatul metodelor testate

Metoda	Validation accuracy
KNN	0.2223
Linear SVM pe pixeli raw	0.2048
Extra Trees pe pixeli raw	0.2684
HOG + Linear SVM	0.3026
HOG + Global + Hough + Linear SVM	0.3252
HOG + Global + Hough + Softmax	0.3342
XGBoost cu feature extraction manual	0.4206
XGBoost + PCA	0.3577
CNN custom RGB	0.6881
DeepSignalCNN RGB	0.6987
DeepSignalCNN RGB + EDGE + FREQ ensemble	0.6997
DeepSignalCNN GREEN + Sliding Window 1×7	0.7139
2D-CNN asymmetric pool	0.7558
2D-CNN + 1D-Profile-CNN ensemble	0.7561
2D old + 2D extra + 1D profile ensemble	0.7571

Comparativ cu metodele clasice, modelele CNN au adus o crestere semnificativa de performanta. Cea mai mare imbunatatire a aparut cand modelul a fost adaptat la structura imaginilor prin pooling asimetric si prin folosirea unui ensemble cu modele antrenate diferit.

15 Concluzii

In cadrul proiectului au fost testate mai multe abordari. Modelele simple, precum KNN si Linear SVM pe pixeli raw, au fost utile pentru verificarea pipeline-ului, dar au avut performante slabe. Feature extraction manual prin HOG, LBP, histograme, proiectii, FFT, connected components si Hough a imbunatatit rezultatele, iar XGBoost a fost cea mai buna metoda clasica.

Cea mai mare crestere de performanta a venit din folosirea retelelor neuronale convolutionale antrenate de la zero. Acestea au invatat automat filtre si reprezentari vizuale relevante pentru imaginile cu semnale radio. Cea mai buna solutie a fost ensemble-ul final dintre 2D-CNN old, 2D-CNN extra seed2026 si 1D-Profile-CNN.

Rezultatul final obtinut pe validation a fost:

0.7571
